

Template

Januar 2023

Über arc42

arc42, das Template zur Dokumentation von Software- und Systemarchitekturen.

Template Version 8.2 DE. (basiert auf AsciiDoc Version), Januar 2023

Created, maintained and © by Dr. Peter Hruschka, Dr. Gernot Starke and contributors. Siehe https://arc42.org.

Diese Version des Templates enthält Hilfen und Erläuterungen. Sie dient der Einarbeitung in arc42 sowie dem Verständnis der Konzepte. Für die Dokumentation eigener System verwenden Sie besser die *plain* Version.

1. Einführung und Ziele

Die xCalendar-App steht für eine einfache Terminplanung unter Terminmitgliedern.

Aufgabenstellung

Inhalt

Es soll eine einfache Web-Applikation mit Angular erstellt werden. Diese kommuniziert über eine API, die durch GraphQL vertreten wird, mit einer Datenbank, MariaDB.

Dabei sollen Terminmitglieder sich mit wenig Aufwand und mit simplen Änderungsvorschlägen über einen Termin absprechen können.

Motivation

Mit der xCalendar-App wollen wir das Planen von Terminen mit Freunden, Bekannten und Familienmitgliedern vereinfachen.

Form

Priorität	Description
1	Durch sauberen und getesteten Code sollen Nutzer in der Applikation keine Fehler erfahren müssen
2	Das Modifizieren sollte durch eine sauber Code-Struktur für zukünftige Veränderungen so einfach und übersichtlich wie möglich sein.
3	Durch das Absprechen mit Teammitgliedern sollen Redundanzen und Fehler in der Software vermieden werden.

Qualitätsziele

Inhalt

Die Applikation soll den Nutzern eine einfache Verwaltung und Absprache von Terminen ermöglichen. Dabei sollen Sie keine Fehler erfahren müssen und am besten auch keinen Datenverlust erleiden müssen.

Motivation

Um den Anforderungen der Stakeholder und damit den Nutzern gerecht zu werden, sollten die Ziele dieser bekannt und verfolgt werden.

Form

Qualitätsziele	Szenarios
Geringes Fehlervorkommen bei der Verwendung	Durch sauberen und getesteten Code sollen Nutzer der Applikation keine Fehler erfahren
Ordentliche Code-Struktur	Das Modifizieren sollte für zukünftige Veränderungen so einfach wie möglich sein

2. Randbedingungen

Inhalt

Rahmenbedingungen für das xCalendar-Projekt sind:
Überschneidungen mit dem „LetsEvent“-Projekt sollten weitestgehend vermieden werden. Die finale Webanwendung soll von jedem internetfähigen Gerät aufrufbar sein. Der gesamte Programmcode wird im Angular-Framework geschrieben und fehlerfrei funktionieren. Parallele Verwendung der Seite durch mehrere Nutzer sollte kein Problem darstellen.
Vorgaben für das xCalendar-Projekt sind:

Benutzer der Anwendung sollen nicht in den Kontakt mit Fehlermeldungen kommen, es sei denn der Fehler geht vom Benutzer aus. Die Antwortzeiten der App bleiben in akzeptablem Rahmen. Alle Features, besonders das Kern-Feature (dem Kalender) werden implementiert und laufen ressourcensparend. HTML-Elemente werden vorzugsweise durch den „class“ Tag identifiziert.

Motivation

Die oben genannten Randbedingungen und Vorgaben, lassen etwas Spielraum bezüglich der Umsetzung des Projekts. Es sind keinerlei visuelle Vorgaben geäußert worden. Darüber hinaus ist die Auswahl existierender Frameworks nicht beschränkt und der gesamte Aufbau der Seite ist nicht näher definiert.

Form

Rahmenbedingung/Vorgabe	Erklärung
Überschneidungen mit „LetsEvent“ vermeiden	Das „LetsEvent“-Projekt weißt viele konzeptionelle Ähnlichkeiten zu diesem Projekt auf. Es liegt im Interesse beider Parteien Gleichheiten in Form und Aufbau zu verhindern.
Gesamter Programmcode im Angular-Framework	Im Sinne der Wartbarkeit und Übersichtlichkeit, soll das gesamte Programm unter Verwendung des Angular-Frameworks geschrieben werden
HTML-Elemente werden via „class“-Tag identifiziert	Um unnötiges Suchen des korrekten Tags vorzubeugen, wird einheitlich definiert, dass ausschließlich der „class“-Tag zu verwenden ist
Features werden implementiert und laufen ressourcensparend	Damit jedes Gerät die App verwenden kann ist das Einsparen von Ressourcen wichtig. Aus diesem Grund muss die App, trotz vieler Features ressourcensparend laufen

3. Kontextabgrenzung

Inhalt

Die Kontextabgrenzung grenzt das System gegen alle Kommunikationspartner (Nachbarsysteme und Benutzerrollen) ab. Sie legt damit die externen Schnittstellen fest und zeigt damit auch die Verantwortlichkeit (scope) Ihres Systems: Welche Verantwortung trägt das System und welche Verantwortung übernehmen die Nachbarsysteme?

Differenzieren Sie fachlichen (Ein- und Ausgaben) und technischen Kontext (Kanäle, Protokolle, Hardware), falls nötig.

Motivation

Es ist wichtig genau zu wissen, welche Systemgrenzen es gibt, welche Benutzer die App verwenden werden, welche externen Systeme und Dienste, welche Schnittstellen, Datenquellen und Entwicklungsumgebungen Verwendung finden. Des Weiteren ist klarzustellen, welche Anforderungen es an die Infrastruktur gibt und was potenzielle nichtfunktionale Anforderungen sein können.

Form

Kommunikationsprinzip der xCalendar-App stark vereinfacht skizziert:

AppComponent ➔ Restful API ➔ Datenbank / externer Service

Datenbank / externer Service ➔ Restful-API ➔ AppComponent

Fachlicher Kontext

Benutzer

Es wird zwischen Endbenutzer und Administrator unterschieden. Endbenutzer sind diejenigen, die die Anwendung über einen Webbrowser verwenden. Der Administrator hat erweiterte Berechtigungen zur Verwaltung und Konfiguration der Anwendung.

Nichtfunktionale Anforderungen

Nichtfunktionale Anforderungen setzten sich im Rahmen dieses Projekts aus den Aspekten Sicherheit, Skalierbarkeit und Verfügbarkeit zusammen. Die Sicherheit soll die Datenintegrität schützen und unbefugten Zugriff, auf bspw. die Administratorrolle, verhindern. Die Skalierbarkeit steht in diesem Kontext für die Fähigkeit wachsende Benutzerzahlen zu bewältigen. Die Verfügbarkeit dagegen, steht für die Gewährleistung, dass die Anwendung (fast) immer für alle Benutzer verfügbar ist.

Technischer Kontext

Systemgrenzen

Es ist festzuhalten, dass die Webanwendung „xCalendar“ das zu entwickelnde System ist. Alle Benutzer interagieren über die graphische Benutzeroberfläche im Webbrowser mit der Anwendung. Die Webanwendung ist an externe Systeme und Dienste geknüpft.

Externe Systeme und Dienste

Das Projekt benötigt einen Authentifizierungsdienst, der für die Authentifikation der Nutzer und deren Accounts zuständig ist. Daneben wird ein Dienst zum Versenden von E-Mails benötigt.

Schnittstellen

HTTP wird zur Kommunikation mit den Benutzerbrowsern über das WWW verwendet. Dazu kommt die Kommunikation mit der Datenbank, welche via API realisiert wird. Die Datenbank-Schnittstelle wird also von der API repräsentiert.

Datenquellen

Die Datenbank speichert Benutzerdaten, Einstellungen und andere wichtige Informationen. Die externe API liefert wichtige Daten von Drittanbieterdiensten.

Infrastruktur

Das Projekt stützt sich auf einen Webserver, der die Webanwendung Hostet und diese über das Internet bereitstellt, einen Datenbankserver, der die Datenbank Hostet und die Benutzer- bzw. Anwendungsdaten speichert und einem Authentifizierungsserver, der die Authentifizierung und Autorisierung von Benutzern verwaltet.

Entwicklungsumgebung

Die Entwicklung findet am Computer statt. Hier wird der Programmcode geschrieben und die App getestet. Zusätzlich gibt es ein Versionskontrollsystem, das den Quellcode speichert und die Zusammenarbeit erleichtert (GitHub).

4. Lösungsstrategie

Im nachfolgenden erläutern wir unsere Lösungsstrategien:

4.1 Technologieentscheidungen

Die Software wird im Frontend mit Angular entwickelt, da diese uns einerseits die Möglichkeit gibt es für den Desktop und das Handy zu entwickeln. Zudem ist das Framework den meisten aus unserem Team bereits bekannt.

Im Backend setzen wir auf GraphQL, welches David in seiner Präsentation vorgestellt hat. Sowie als Datenbank SQL.

4.2 Architekturmuster

Dieses Projekt verwendet das MVC-Entwicklungsmuster, sowohl für Frontend als auch Backend insgesamt.

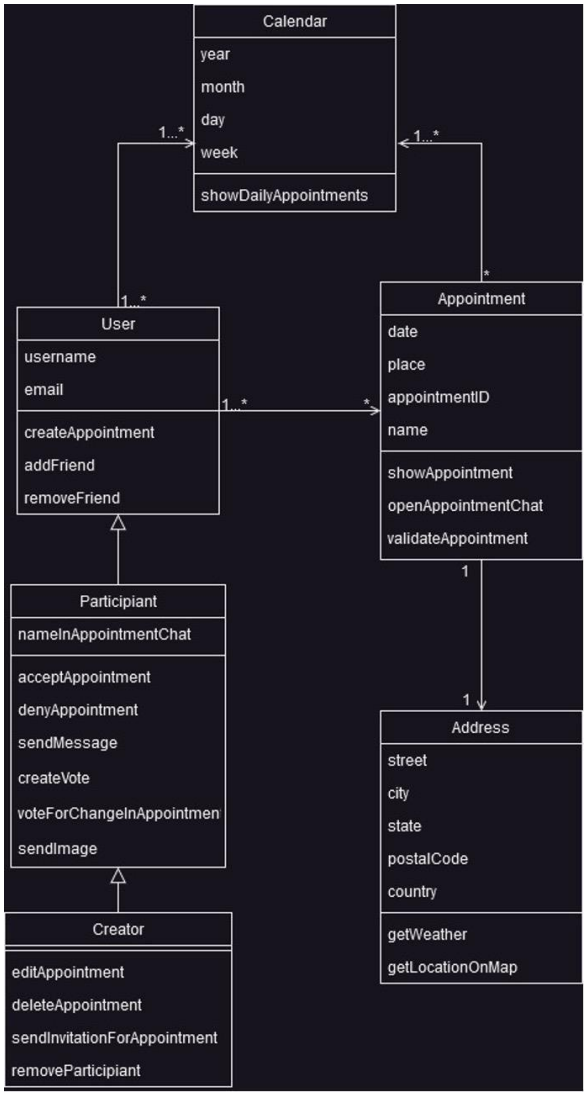
4.3 Erreichen wichtige Qualitätsmerkmale

Die Software wird regelmäßig gebaut von unserem Jenkins um gröbere Fehler zu vermeiden zu dem wird eine Live Demo dauerhaft bereitstehen um Feature durch testen zu können ohne großen Aufwand.

4.4 Organisatorische Entscheidungen

Die Dokumentation des Projektmanagement findet man in JIRA. Hier sind alle Aufgaben und Task dokumentiert.

5. Bausteinsicht



Creator: Terminersteller (Erbt von Participiant)

Participiant: Terminteilnehmern (Erbt von User)

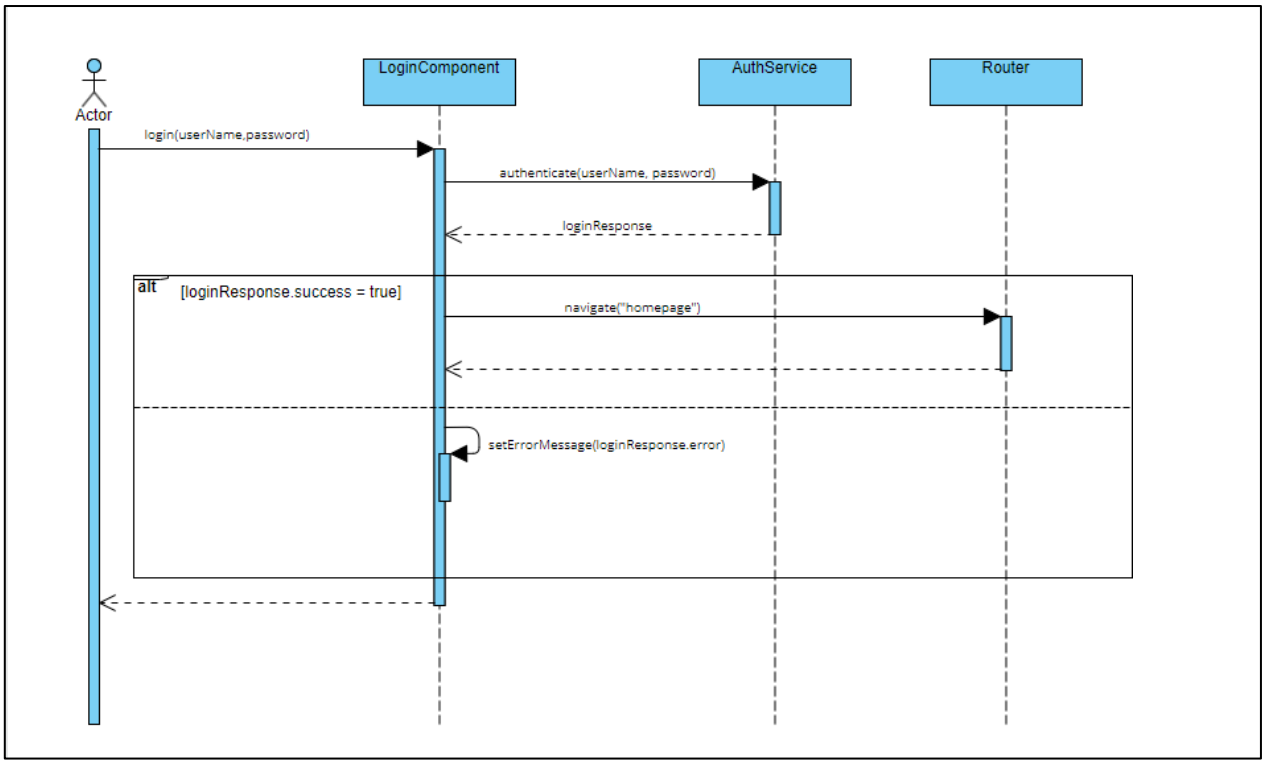
User: xCalendar-Benutzer (Es gibt mehrere Benutzer pro Terminkalender und Termin)

Calendar: Terminkalender

Appointment: Einzelner Termin (Es gibt mehrere Termine in einem Terminkalender)

Adress: Adresse für ein Termin (Es gibt eine Adresse/einen Ort für einen Termin)

6. Laufzeitsicht



Login-Anfrage initiieren:

Der Benutzer startet die Anmeldung, indem er Aktionen in der Benutzeroberfläche ausführt (z. B. Klicken auf einen Anmeldebutton).

Die Benutzeroberfläche sendet eine Anforderung an den "LoginComponent".

LoginComponent:

Der "LoginComponent" empfängt die Anfrage und leitet sie an den "AuthService" weiter.

Der "LoginComponent" wartet auf die Antwort des "AuthService".

AuthService - Authentifizierung:

Der "AuthService" empfängt die Anmeldeanforderung vom "LoginComponent".

Der "AuthService" führt die Authentifizierung durch (Überprüfung von Benutzername und Passwort, zum Beispiel).

Der "AuthService" gibt eine Antwort zurück, die den Authentifizierungsstatus enthält (erfolgreich oder fehlgeschlagen) und ggf. Benutzerinformationen.

LoginComponent - Benachrichtigung über Authentifizierungsstatus:

Der "LoginComponent" empfängt die Antwort des "AuthService".

Basierend auf dem Authentifizierungsstatus aktualisiert der "LoginComponent" die Benutzeroberfläche entsprechend (z. B. Anzeige einer Erfolgsmeldung oder Fehlermeldung).

Router - Navigation:

Falls die Authentifizierung erfolgreich war, könnte der "Router" benachrichtigt werden.

Der "Router" entscheidet, welche Ansicht oder Seite als nächstes angezeigt werden soll (z. B. die Hauptseite nach erfolgreicher Anmeldung).

7. Verteilungssicht

Inhalt

Der xCalendar wird über eine Webseite für alle erreichbar sein. Diese Webseite ist von allen Geräten mit Browsern aufrufbar.

Die Webseite läuft über einen simplen WebServer im unseren Test Server Fall auf Ubuntu mit dem Apache2 WebServer.

Das gebaute Angular Projekt muss also nur auf einem Web Server deployt werden und mit einer entsprechenden SQL-Datenbank verbunden schon läuft das Projekt. Als Benutzer muss man sich diese Gedanken allerdings nicht machen, da dies von uns bereitgestellt wird. Der Benutzer benötigt also lediglich ein Gerät seiner Wahl welches einen Browser hat.

8. Querschnittliche Konzepte

Inhalt

Als konzeptionelle Architektur, entschieden wir uns für das „Model-View-Controller“ (MVC)-Modell. Damit haben wir dann drei Architekturebenen, in die unser Projekt aufgeteilt ist.

Es werden Technologien verwendet, die eine schnelle Antwortzeit liefern, und einfach zu warten sind, falls es mal zu Fehlern kommen sollte.

Motivation

Unsere Motivation dafür ist, ein zufriedener Nutzer, der vom Design und der Funktionalität der Software überzeugt ist.

Form

Angular ist das Modell unserer Software, die mithilfe der TypeScript-Sprache als eine Art „Gerüst“ für die Darstellung („View“) und das Steuern der Daten („Controller“) dient.

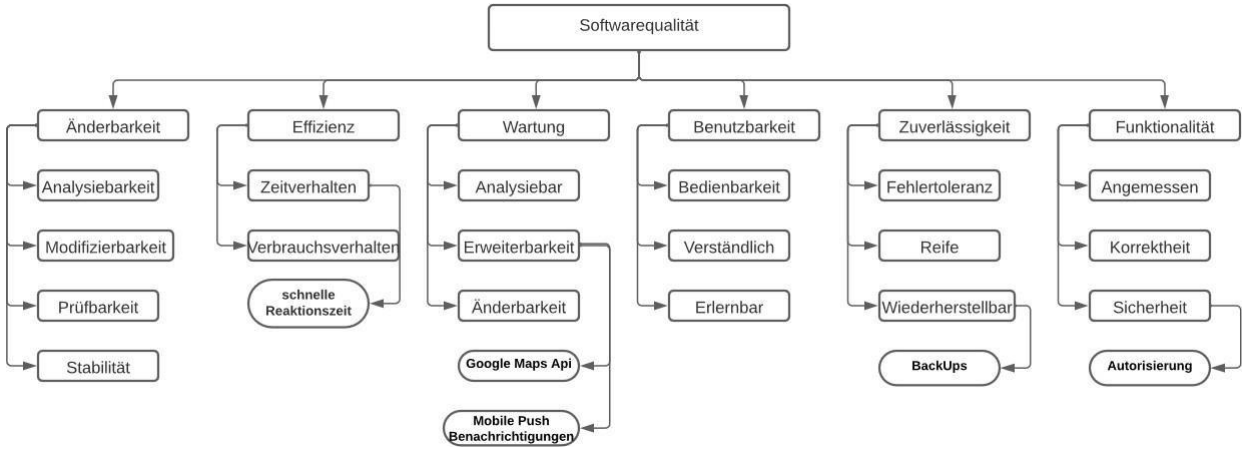
In diesem Gerüst werden die Bausteine von „Material Design“, als Frontend-Framework, und von GraphQL, eine „Query Language“ (Abfragesprache) für API’s, eingesetzt.

9. Architekturentscheidungen

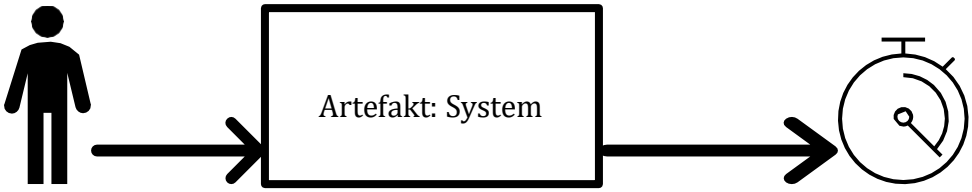
- Frontend Angular
- Backend GraphQL
- Datenbanken MariaDB SQL

10. Qualitätsanforderungen

Qualitätsbaum



Qualitätsszenarien



Quelle:	Stimulus:	Antwort:	Antwortmaß:
Benutzer	erstellt neuen Termin	System speichert Termindaten	Durchschnittliche Latenzzeit von 1 Sekunde

11. Risiken und technische Schulden

Architekturrisiken	Risiko	Maßnahmen	Priorität
Datensicherheit und Datenschutz	Mögliche Verletzungen der Privatsphäre durch unsachgemäße Handhabung von Benutzerdaten	Implementierung von robusten Sicherheitsmechanismen, Datenschutzrichtlinien und regelmäßige Sicherheitsaudits.	1
Skalierbarkeit	Unzureichende Skalierbarkeit der Anwendung bei steigender Benutzerzahl.	Implementierung skalierbarer Architektur, Lasttests und Ressourcenoptimierung.	2
Integration von Drittanbieterdienste	Probleme bei der nahtlosen Integration von externen Diensten (z.B., Wetterdaten).	Frühzeitige Tests, regelmäßige Überprüfung von APIs und alternative Lösungsansätze.	3

Technische Schulden	Schulden	Abbau	Priorität
Veraltete Bibliotheken und Frameworks	Verwendung veralteter Technologien kann zu Sicherheitsproblemen und eingeschränkter Funktionalität führen	Plan für das schrittweise Aktualisieren von Bibliotheken und Frameworks erstellen, um auf dem neuesten Stand zu bleiben.	1
Code-Duplizierung	Duplizierter Code führt zu Wartbarkeitsproblemen und erschwert zukünftige Entwicklungen.	Identifizierung und Refaktorisierung von dupliziertem Code während des Entwicklungszyklus.	2
Unzureichende Dokumentation	Fehlende oder unklare Dokumentation beeinträchtigt die Zusammenarbeit und zukünftige Entwicklungen.	Regelmäßige Aktualisierung von Entwicklungs- und Architekturdokumentation sowie Schulungen für das Entwicklerteam.	3

12. Glossar

Begriff	Definition	Übersetzung
Authentifizierung	Der Prozess der Überprüfung der Identität eines Benutzers, um den Zugriff auf das System zu gewähren.	Authentication
App Komponente	Einzelne funktionale Einheit oder Module innerhalb der Anwendung, die spezifische Aufgaben erfüllen.	App Component
Benutzerprofil	Beschreibung der Eigenschaften und Rechte eines Benutzers im System.	User Profile
Terminfreigabe	Die Funktion, Termine mit Freunden zu teilen und gemeinsam zu planen.	Appointment Sharing
Chat-Funktionalität	Die Möglichkeit, in Echtzeit mit anderen Benutzern zu kommunizieren.	Chat Functionality
Wetteranzeige	Die Darstellung aktueller Wetterinformationen in der Anwendung.	Weather Display
Skalierbarkeit	Die Fähigkeit des Systems, mit steigender Benutzerzahl effizient umzugehen.	Scalability
Datenschutzrichtlinien	Festgelegte Regeln und Maßnahmen zum Schutz der Privatsphäre der Benutzer.	Privacy Policies
API-Integration	Die Einbindung externer Dienste über Application Programming Interfaces.	API Integration
Code-Refaktorisierung	Umstrukturierung des Quellcodes zur Verbesserung von Lesbarkeit und Wartbarkeit.	Code Refactoring